PA-HGS: Phase-Aware Heterogeneous GPU Scheduling for Energy–Latency Optimization

Alexander Lee
Department of Electrical Engineering
San José State University
San José, CA, USA
alexander.h.lee@sjsu.edu

***Abstract*— The increasing use of deep-learning and high-performance-computing workloads has intensified the energy and performance challenges of GPU clusters. These challenges are amplified in heterogeneous environments, where GPUs differ in computational capability, memory performance, power consumption, and frequency behavior. This paper proposes a Phase-Aware Heterogeneous GPU Scheduler (PA-HGS) that jointly selects GPU placement, operating frequency, and queue position for each workload phase under a fixed cluster-wide power budget and latency constraint. The framework models compute-bound, memory-bound, and mixed phases and adapts frequency and GPU selection at phase boundaries according to predicted execution time, power consumption, queueing delay, and GPU suitability. It also accounts for backfilling opportunities, frequency-transition overhead, and GPU migration costs. An event-driven simulator evaluates PA-HGS against backfilling, workload-aware scheduling, and DVFS-only scheduling using identical heterogeneous GPU configurations and workload traces. The simulation estimates energy-to-solution, total energy, job completion time, queueing delay, energy-delay product, throughput, GPU utilization, and deadline-violation rate. A 5% completion-time degradation limit is applied to evaluate energy savings without unacceptable performance loss. The modeled results indicate that DVFS-only scheduling generally minimizes total energy, whereas PA-HGS provides a better energy–latency tradeoff for memory-bound and mixed workloads. PA-HGS achieves the lowest deadline-violation rate across the evaluated workload classes and reduces energy-delay products for memory-bound and mixed workloads.**

Keywords*— *GPU clusters, energy efficiency, workload-aware scheduling, heterogeneous computing, dynamic voltage and frequency scaling, GPU power management, latency optimization.

I. Introduction

The rapid adoption of large language models and other deep-learning applications has increased demand for GPU-based computing and data-center energy. Training and inference workloads differ in computational intensity, memory behavior, and execution time, while GPU platforms differ in computational capability, memory bandwidth, power consumption, and frequency behavior [1], [2], [3]. These differences make energy-efficient management difficult in heterogeneous GPU clusters, where a scheduling decision that benefits one workload may increase energy consumption or latency for another.

Existing GPU-cluster schedulers commonly optimize job completion time, fairness, throughput, utilization, or resource fragmentation. Backfilling can improve utilization by executing later jobs in temporarily available resource gaps, while workload-aware and heterogeneity-aware methods attempt to match jobs with suitable resources [4], [5], [6], [7]. Other studies use Dynamic Voltage and Frequency Scaling (DVFS), power capping, runtime power configuration, or thermal-aware scheduling to reduce energy consumption and control GPU operating conditions [8], [9]. Although these approaches demonstrate the value of workload-aware placement and power management, they often evaluate scheduling, GPU assignment, and frequency control separately.

These decisions are closely coupled at the cluster level. Assigning a workload to an energy-efficient GPU may increase queueing delay if that device is occupied, while reducing the frequency of a compute-intensive phase may lower instantaneous power but significantly increase execution time. GPU selection and frequency decisions can also affect other jobs through cluster-wide power limits, resource contention, frequency-transition overhead, and migration costs. Therefore, energy efficiency must be evaluated together with execution time, job completion time, queueing delay, energy-to-solution, and deadline compliance.

This paper proposes a cluster-level Phase-Aware Heterogeneous GPU Scheduler (PA-HGS) for energy- and latency-aware scheduling in heterogeneous GPU clusters. For each schedulable workload phase, PA-HGS jointly selects a GPU, an operating frequency, and a queue position. These decisions consider phase characteristics, predicted execution time, estimated power consumption, GPU suitability, queueing delay, and current cluster resource availability. The scheduler adapts GPU and frequency assignments at phase boundaries and considers backfilling opportunities when they satisfy the cluster-wide power budget and completion-time constraint. PA-HGS therefore extends phase-aware DVFS by integrating frequency control with heterogeneous GPU placement and queue management.

The framework evaluates PA-HGS against backfilling, workload-aware scheduling, and DVFS-only scheduling under consistent workload traces, GPU configurations, power parameters, and scheduling constraints. Compute-bound, memory-bound, and mixed workloads are represented using analytical execution-time and power models and evaluated using an event-driven simulation. The evaluation considers energy-to-solution, total cluster energy, job completion time, queueing delay, energy-delay product, throughput, GPU

utilization, scheduling overhead, and deadline-violation rate. A bounded completion-time degradation threshold is applied to determine whether energy savings can be achieved without unacceptable performance loss.

The purpose of this study is not to claim that one scheduling policy is universally optimal. Instead, it examines the conditions under which jointly selecting GPU placement, operating frequency, and queue position improves the modeled energy–latency tradeoff. The analysis evaluates how workload phases, GPU heterogeneity, frequency selection, queueing decisions, power-budget enforcement, and scheduling overhead affect cluster performance. Because the evaluation relies on analytical models and event-driven simulation, the results represent modeled estimates under the stated workload, hardware, and latency assumptions rather than direct measurements from a physical GPU cluster.

II. RESEARCH BACKGROUND

GPU scheduling and cluster management have evolved from simple resource allocation toward multi-objective strategies that jointly consider utilization, energy consumption, thermal conditions, and service performance. Thermal-aware scheduling studies use temperature prediction to distribute GPU workloads across cooler hosts, reducing computing costs by up to 12.79% compared with baseline policies [8]. Other work combines power-aware scheduling with fragmentation management in Kubernetes, achieving power savings of up to 20% compared with fragmentation-aware scheduling alone [6]. A constrained reinforcement-learning scheduler for heterogeneous, geographically distributed data centers has also combined data-center routing, GPU-count selection, and adaptive frequency scaling, improving job completion speed while reducing energy per unit of work [10]. These studies demonstrate that GPU cluster management must consider power, thermal state, resource fragmentation, workload behavior, and service-level requirements rather than relying only on GPU utilization or arrival order.

GPU Dynamic Voltage and Frequency Scaling (DVFS) provides a mechanism for adjusting GPU performance and power according to workload demands. However, the energy and performance benefits of a frequency setting depend on the workload, GPU architecture, memory behavior, and latency requirement. Latency-aware GPU DVFS has been used to reduce energy while satisfying service-level constraints [11], while runtime power-management systems use feedback control and frequency adjustment to maintain a target power level under changing workloads [12]. Other DVFS studies develop performance and power models that account for execution time, memory behavior, and frequency-dependent energy consumption [13], [14], [15]. These studies motivate the use of workload-dependent power and execution-time models in this paper, although DVFS is treated as one component of a broader scheduling problem rather than as an isolated optimization technique.

A more fine-grained approach is phase-aware scheduling. Instead of assigning one static scheduling or frequency decision to an entire job, phase-aware methods recognize that an application can transition between execution phases with different computational and memory requirements. For example, a job may alternate between compute-intensive, memory-bound, communication, and synchronization phases. A frequency that is effective during a compute-intensive phase may waste energy during a memory-bound phase because increasing the core frequency provides limited performance improvement when memory access is the dominant bottleneck. Consequently, phase-aware scheduling attempts to identify the current or upcoming phase and select an appropriate resource configuration.

Previous research has demonstrated several approaches for phase-aware DVFS. Program phase-aware DVFS methods detect recurring program phases and account for both computational workload and memory-stall behavior when selecting operating frequencies [16]. More recent work applies phase-based frequency scaling to heterogeneous multi-GPU systems by using application profiling and task-graph information to identify energy-related phases and assign frequency settings to individual phases. These methods also consider the overhead associated with changing frequency between phases [17]. For latency-sensitive GPU inference, iteration-level DVFS adjusts the frequency during execution to reduce energy while satisfying service-level objectives and tail-latency requirements [18]. Phase-aware placement and DVFS frameworks for disaggregated language-model serving further demonstrate that different inference phases can have different hardware affinities, queueing behavior, and energy-performance tradeoffs [19].

Phase-aware scheduling is therefore related to workload-aware scheduling but operates at a finer temporal granularity. Workload-aware scheduling may classify an entire job as compute-bound or memory-bound before execution, whereas phase-aware scheduling updates resource decisions as the workload transitions between phases. This distinction is important for heterogeneous GPU clusters because both the best GPU type and the best frequency may change during execution. However, existing phase-aware approaches often focus on frequency selection, task placement, or specialized inference pipelines separately. They do not always consider queue position, backfilling decisions, cluster-wide power limits, and latency constraints within one scheduling decision.

Heterogeneous GPU scheduling remains important because modern clusters combine GPUs with different computational capabilities, memory systems, power characteristics, and resource-sharing behavior. Studies of fine-grained heterogeneous execution and HeShare show that task-to-GPU assignment, GPU sharing, and DVFS can be jointly used to balance performance and energy consumption [20], [21], [22]. Deep-learning-based placement research further shows that workload interference can vary according to computation, memory, CPU, storage, and network demands. These findings indicate that phase-aware decisions should account for both the characteristics of the current workload phase and the capabilities of the available GPU.

The reviewed studies establish the importance of phase behavior, heterogeneous hardware, DVFS, queueing, and power constraints, but they generally evaluate these factors under different workloads, hardware platforms, and objectives.

This makes direct comparison difficult and leaves an opportunity for a common cluster-level evaluation framework.

Therefore, this paper proposes a phase-aware heterogeneous GPU scheduling policy that jointly selects workload queue position, GPU assignment, and DVFS state. The policy evaluates these decisions under a fixed cluster-wide power budget and a bounded latency constraint. An event-driven simulation is then used to compare the proposed policy with backfilling, workload-aware scheduling, and DVFS-only scheduling across compute-bound, memory-bound, and mixed workloads using consistent energy, latency, throughput, utilization, and deadline-violation metrics.

III. TECHNICAL ANALYSIS

This study develops an event-driven simulation framework to evaluate energy- and latency-aware scheduling in a heterogeneous GPU cluster. The framework compares scheduling policies under identical workload traces, GPU configurations, power limits, and latency requirements. The objective is to determine whether the proposed Phase-Aware Heterogeneous GPU Scheduler (PA-HGS) improves the energy–latency tradeoff by jointly selecting a workload's queue position, GPU type, and operating frequency.

The simulated cluster contains multiple GPU types with different computational capabilities, memory-performance characteristics, idle power levels, and dynamic power consumption. These differences represent the hardware heterogeneity considered in prior GPU scheduling research [14], [15], [20]. Each job is represented by its arrival time, workload class, execution phases, computational demand, memory demand, and latency requirement.

Each job contains multiple execution phases classified as compute-intensive, memory-intensive, or mixed. Compute-intensive phases benefit more from increased GPU core frequency, whereas memory-intensive phases receive smaller performance improvements because their execution is primarily limited by memory behavior. This phase-based representation allows the simulator to evaluate scheduling decisions at a finer granularity than a policy that assigns one GPU or frequency configuration to an entire job.

The simulation assumes that phase characteristics are known or accurately estimated before scheduling. This assumption isolates the scheduling and resource-allocation problem. The effects of inaccurate phase prediction are discussed as a limitation and future research direction.

The execution time of job i during phase p, when assigned to GPU type g at normalized frequency f, is modeled as

$$T_{i,p}(g,f) = \frac{W_{i,p}}{P_g S_p(f)} \tag{1}$$

where $W_{i,p}$ is the amount of work in phase p, P_g is the normalized performance factor of GPU type g, and $S_p(f)$ is the frequency-dependent speedup factor for the phase type.

The speedup function differs according to the phase classification. Compute-bound phases receive a larger performance benefit from increased frequency, while memory-bound phases receive a smaller benefit. Mixed phases use an intermediate scaling relationship. Consequently, the same frequency setting can produce different execution-time and energy outcomes depending on the current phase.

GPU power is modeled as the sum of idle power and workload-dependent dynamic power:

$$P_{i,g}(f) = P_{idle,g} + P_{dynamic,g} A_p F(f) \tag{2}$$

where $P_{idle,g}$ and $P_{dynamic,g}$ are the idle and dynamic power parameters of GPU type g, A_p is the activity factor associated with the phase type, and $F(f)$ represents the frequency-dependent dynamic-power relationship.

The energy consumed by a phase is calculated as

$$E_{i,p} = P_{i,g}(g,f) T_{i,g}(g,f) \tag{3}$$

and the total energy required by a job is

$$E_i = \sum_p E_{i,p} \tag{4}$$

The power and performance parameters are based on published GPU measurements and analytical GPU power models [11], [13], [15]. Because the evaluation is simulation-based, these values represent controlled modeling assumptions rather than direct measurements from a physical cluster.

The current simulation models one active job phase per GPU. Detailed multi-tenant memory-bandwidth contention, PCIe contention, thermal throttling, and network interference are not explicitly modeled. These factors could be incorporated in future work through an interference factor that increases execution time when incompatible workloads are collocated.

A. Phase-Aware Heterogeneous Scheduling

At each scheduling event, PA-HGS examines newly arrived jobs, completed phases, available GPUs, and the current cluster power state. For each eligible job, GPU, frequency, and queue-position combination, the scheduler estimates phase execution time, energy consumption, completion time, and deadline feasibility.

The policy considers the current phase type, the performance and power characteristics of each GPU, the available frequency levels, the position of the job in the queue, the cluster's current power consumption, and the job's latency requirement. Compute-intensive phases are more likely to be assigned to high-performance GPUs or higher frequencies when the resulting latency benefit justifies the additional energy. Memory-intensive phases are more likely to use lower-frequency configurations or GPUs that provide better energy efficiency for memory-limited execution.

PA-HGS may also advance a later job in the queue when doing so improve system efficiency without causing an unacceptable delay for earlier jobs. This allows queue position to be evaluated jointly with GPU assignment and frequency selection rather than treated as an independent scheduling decision.

A candidate configuration is feasible only when it satisfies the cluster power constraint,

$$P_{cluster}(t) \leq P_{budget} \quad (5)$$

and the latency requirement,

$$T_i \leq T_{i,baseline}(1 + \delta) \quad (6)$$

where (P_{budget}) is the fixed cluster power budget, $(T_{i,baseline})$ is the baseline completion-time estimate, and δ is the maximum permitted performance degradation. In this evaluation, δ is set to 0.05 when the performance-degradation constraint is applied.

Among feasible configurations, PA-HGS selects the option with the lowest combined scheduling cost. This cost incorporates estimated energy, energy-delay products, queueing delay, predicted latency violation, and the suitability of the selected GPU for the current phase. The policy therefore balances energy efficiency and service performance instead of minimizing only one metric.

The energy-delay product is calculated as

$$EDP_i = E_i T_i \quad (7)$$

where E_i is the total energy consumed by job i and T_i is its completion time. A lower EDP indicates a more favorable combined energy and latency outcome.

B. Scheduling Policies

The simulation compares PA-HGS with backfilling, workload-aware scheduling, and DVFS-only scheduling. Backfilling improves GPU utilization by executing later jobs in temporarily available resource gaps when doing so does not excessively delay jobs already selected for execution.

Workload-aware scheduling classifies jobs according to their dominant compute or memory behavior and uses this information to reduce incompatible resource contention. However, it does not dynamically optimize every execution phase's GPU and frequency configuration.

DVFS-only scheduling selects frequency settings to reduce power and energy consumption but does not jointly optimize queue position or GPU assignment according to the current execution phase. In contrast, PA-HGS jointly selects queue position, GPU type, and frequency for each schedulable phase while considering the cluster power budget and latency constraint.

This comparison separates the effects of queue management, workload classification, frequency scaling, GPU heterogeneity, and phase-level adaptation. It also allows the proposed policy to be compared with policies that optimize only one or two of these factors.

C. Event-Driven Simulator Procedure

The simulator advances between discrete scheduling events rather than using a fixed simulation time step. Events include job arrivals, phase completions, frequency transitions, GPU availability changes, and scheduling decisions. At each event, the simulator updates the current time, records energy consumed since the previous event, identifies completed jobs and phases, updates GPU availability and cluster power, and adds newly arrived jobs to the queue.

After the system state is updated, the active policy evaluates the schedulable jobs and available GPU configurations. The selected decisions determine which phases begin execution, which GPU and frequency are used, and when the next phase-completion or job-arrival event will occur. Frequency-transition and migration overheads are included as additional time and energy costs when a job changes frequency or GPU assignment.

All policies receive the same job arrivals, workload phases, GPU inventory, power parameters, and latency requirements. The experiments use compute-bound, memory-bound, and mixed workload traces, and the results are averaged across multiple random seeds to reduce the influence of any individual synthetic trace.

D. Evaluation Metrics

The policies are evaluated using energy, performance, utilization, and service-quality metrics. Energy-to-solution measures the energy required to complete an individual job, while total cluster energy measures energy consumption over the complete simulation interval. Job completion time includes both queueing delay and execution time. The evaluation also measures average latency, 95th-percentile latency, energy-delay product, throughput, GPU utilization, deadline-violation rate, and scheduling overhead.

The analysis examines whether phase classification reduces energy consumption compared with backfilling, whether heterogeneous GPU assignment improves completion time or energy-to-solution, and whether DVFS provides different benefits for compute-bound and memory-bound phases. It also evaluates whether jointly selecting queue position, GPU type, and frequency improves EDP and reduces deadline violations under a fixed cluster power budget.

Because this study uses analytical models and event-driven simulation rather than measurements from a physical GPU cluster, the results represent modeled outcomes under the stated assumptions. The purpose of the analysis is not to claim that one policy is universally optimal, but to identify the workload and hardware conditions under which phase-aware scheduling provides a favorable energy–latency tradeoff.

IV. Results/Evaluation

A. Experimental Configuration

The proposed scheduler was evaluated using the event-driven heterogeneous GPU-cluster simulator described in Section III. The simulated cluster contained two A100 GPUs and two T4 GPUs. Each GPU supported normalized frequency levels of 0.60, 0.80, and 1.00. The cluster-wide power budget was fixed at 330 W, and a latency violation was recorded when a job completion time exceeded its modeled deadline by more than 5%.

Three workload classes were evaluated: compute-bound, memory-bound, and mixed workloads. Each job contained two to four execution phases, with each phase assigned between 18 and 42 abstract work units. Compute-bound jobs primarily contain compute phases, memory-bound jobs primarily contain memory phases, and mixed jobs contain a balanced combination of compute, memory, and mixed phases. Job arrivals followed

an exponential distribution with a rate of 0.06 jobs per second, corresponding to a mean interarrival time of approximately 16.67 s.

Each experiment used the same workload trace for every scheduling policy. Results were averaged over ten random seeds. The evaluated policies were backfilling, workload-aware scheduling, DVFS-only scheduling, and the proposed phase-aware heterogeneous policy.

TABLE I. GPU CONFIGURATION

GPU Type	Quantity	Normalized Performance	Freq. Levels	Idle Power	Max Power
A100	2	4.00	0.60, 0.80,1.00	35W	230W
T4	2	1.50	0.60, 0.80,1.00	12W	55W

TABLE II. WORKLOAD AND SIMULATION PARAMETERS

Category	Parameter	Setting
Cluster	Total power budget	330W
Workloads	Workload classes	Compute-bound, memory-bound, mixed
Workloads	Phases per job	2-4
Workloads	Work per phase	18-42 abstract work units
Arrivals	Arrival distribution	Exponential
Arrivals	Mean interarrival time	16.67s
Evaluation	Random seeds	10 independent seeds
Evaluation	Scheduling policies	Backfilling, workload-aware, DVFS-only, PA-HGS
Constraint	Latency threshold	5% deadline-degradation limit

The simulated cluster consisted of two A100 GPUs and two T4 GPUs. The GPU-specific performance and power parameters are summarized in Table I, while the workload-generation and simulation parameters are listed in Table II.

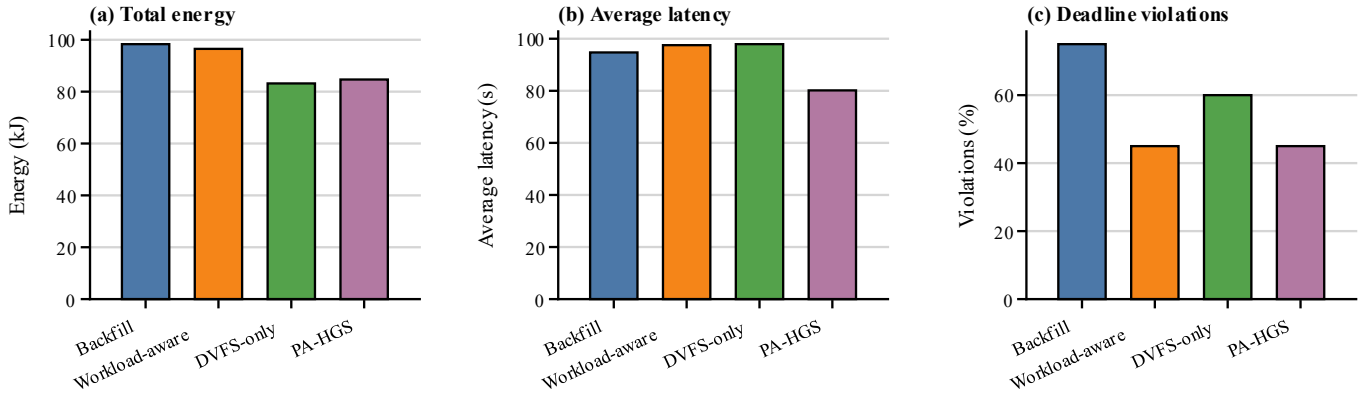


Fig 1. Simulated Results of total energy, average latency, and deadline-violation rate for backfilling, workload-aware, DVFS-only, and PA-HGS scheduling.

Fig. 1 provides an overall comparison of the evaluated scheduling policies across total energy consumption, average latency, and deadline-violation rate. The results show that no policy consistently optimizes all three metrics. DVFS-only scheduling generally achieves the lowest energy consumption because it reduces operating frequency and dynamic power. However, this energy reduction can increase execution time and deadline violations, particularly for compute-bound workloads.

PA-HGS provides a more balanced energy–latency outcome. Although it does not achieve the lowest total energy in every workload class, it achieves the lowest deadline-violation rate for compute-bound, memory-bound, and mixed workloads. Its strongest performance occurs for memory-bound and mixed workloads, where phase-aware GPU placement and frequency selection reduce latency without producing a substantial increase in energy.

Backfilling provides a useful resource-utilization baseline but does not explicitly account for workload phase behavior, GPU heterogeneity, or power consumption. Workload-aware scheduling improves deadline compliance relative to backfilling in several cases, but it does not jointly optimize GPU placement and frequency selection. These overall trends motivate the workload-specific analysis presented in the following subsections.

B. Compute-Bound Workloads

TABLE III. COMPUTE-BOUND WORKLOAD RESULTS

Policy	Energy(kJ)	Avg. latency(s)	Queue delay(s)	Throughput	Utilization	EDP (MJ·s)	Violation rate

Backfilling	428.4	127.2	78.3	4.823	66.9%	54.53	57.4%
Workload-aware	428.9	146.7	95.9	4.778	69.0%	62.99	35.5%
DVFS-only	390.3	141.8	89.3	4.716	70.2%	55.30	65.2%
PA_HGS	398.1	142.8	93.8	4.838	67.3%	56.81	30.2%

For compute-bound workloads, the DVFS-only policy achieved the lowest energy consumption at 390.3 kJ because reducing frequency lowered dynamic power. However, its latency violation rate was relatively high because lower frequencies increased execution time. The proposed policy consumed approximately 7.1% less energy than backfilling while achieving the highest throughput of 4.838 work units per second. It also produced the lowest deadline-violation rate, at 30.2%, which was approximately 14.9% lower than the workload-aware policy and 53.7% lower than the DVFS-only policy.

The proposed policy did not achieve the lowest average latency for compute-bound jobs. This occurs because compute-bound phases benefit strongly from the higher-performance A100 GPUs and higher frequencies, whereas the proposed policy sometimes selects lower-power configurations to satisfy the cluster power budget. These results illustrate the tradeoff between energy reduction and latency for compute-intensive workloads.

C. Memory Bound Workloads

TABLE IV. MEMORY-BOUND WORKLOADS

Policy	**Energy(kJ)**	**Avg. latency(s)**	**Queue delay(s)**	**Throughput**	**Utilization**	**EDP (MJ·s)**	**Violation rate**
Backfilling	380.0	85.6	40.6	5.079	64.8%	32.47	33.0%
Workload-aware	381.0	90.3	45.6	5.100	64.7%	34.31	26.0%
DVFS-only	289.7	58.7	19.4	5.213	58.0%	16.99	12.1
PA-HGS	291.0	50.8	14.7	5.243	53.7%	14.80	7.7%

The proposed policy performed most effectively for memory-bound workloads. Its average latency was 50.8 s, representing a reduction of approximately 13.4% compared with DVFS-only scheduling and 40.6% compared with backfilling. The proposed policy also reduced queueing delay to 14.7 s and achieved the highest throughput at 5.243 work units per second.

DVFS-only scheduling produced slightly lower energy consumption than the proposed policy, with 289.7 kJ compared with 291.0 kJ. However, the proposed policy achieved a lower EDP because it reduced latency more substantially. Its EDP was 14.80 MJ·s, approximately 12.9% lower than DVFS-only scheduling. The proposed policy also reduced the deadline-violation rate from 12.1% to 7.7%.

These results are consistent with the modeled behavior of memory-bound workloads. Increasing GPU frequency provides limited performance improvement for memory-bound phases, so the proposed policy can use lower-frequency configurations without substantially increasing execution time. This allows it to reduce energy while preserving latency.

D. Mixed Workloads

TABLE V. MIXED WORKLOADS

Policy	**Energy(kJ)**	**Avg. latency(s)**	**Queue delay(s)**	**Throughput**	**Utilization**	**EDP (MJ·s)**	**Violation rate**
Backfilling	410.8	115.5	67.4	4.986	65.7%	47.21	39.9%
Workload-aware	409.0	121.5	73.2	5.005	66.4%	49.46	29.4%
DVFS-only	347.0	123.1	73.7	4.933	66.9%	42.67	44.5%
PA-HGS	351.4	105.7	60.2	5.066	63.2%	37.04	19.0%

For mixed workloads, the proposed policy achieved the best overall energy–latency balance. Its average latency was 105.7 s, which was approximately 8.5% lower than backfilling and 14.1% lower than workload-aware scheduling. It also achieved the highest throughput at 5.066 work units per second.

DVFS-only scheduling produced the lowest energy consumption at 347.0 kJ, while the proposed policy consumed 351.4 kJ, representing an energy increase of approximately 1.3%. However, the proposed policy reduced EDP by approximately 13.2% compared with DVFS-only scheduling because of its lower latency. The proposed policy also achieved the lowest deadline-violation rate, reducing violations to 19.0%, compared with 44.5% for DVFS-only scheduling.

The improvement for mixed workloads results from the policy's ability to distinguish between compute-sensitive and memory-sensitive phases. Compute phases can be assigned to high-performance A100 GPUs when additional throughput is needed, while memory-sensitive phases can use lower-frequency or lower-power configurations. This phase-level adaptation allows the proposed policy to balance energy consumption and completion time more effectively than

policies that apply one scheduling or frequency decision to the entire job.

E. Cross-Workload Comparison

The results show that no policy was uniformly optimal for every metric and workload class. DVFS-only scheduling generally minimizes total energy, particularly for memory-bound workloads, but it sometimes increased queueing delay and latency violations. Backfilling provided a simple mechanism for improving resource utilization but did not explicitly account for workload characteristics or GPU energy behavior. Workload-aware scheduling reduced deadline violations for some workload classes but used the maximum frequency and therefore did not consistently minimize energy.

The proposed policy achieved its strongest performance for memory-bound and mixed workloads. Compared with the best competing policy for each workload class, it reduced EDP by approximately 12.9% for memory-bound workloads and 13.2% for mixed workloads. It also achieved the lowest deadline-violation rate for all three workload classes. For compute-bound workloads, the proposed policy reduced energy relative to backfilling but did not minimize EDP or average latency. This limitation occurs because compute-bound phases benefit from high-frequency execution, and aggressive energy reduction can increase completion time.

These results support the central hypothesis that jointly considering workload phase, GPU heterogeneity, DVFS state, queue position, and the cluster power budget can improve energy–latency efficiency compared with policies that optimize only one of these factors. However, the results should be interpreted as model-based estimates. The numerical outcomes depend on the assumed GPU performance factors, power models, arrival distribution, phase lengths, transition overheads, and deadline formulation.

V. Discussion

The modeled results show that no single scheduling or power-management strategy provides the best outcome for every workload class and metric. DVFS-only scheduling generally achieved the lowest total energy because it reduced operating frequency and dynamic power. However, this energy reduction sometimes increased execution time and deadline violations. Backfilling provided a simple mechanism for improving resource utilization, but it did not explicitly account for workload behavior, GPU heterogeneity, or phase-specific power requirements. Workload-aware scheduling reduced deadline violations for some workload classes, but its use of maximum frequency did not consistently minimize energy.

PA-HGS achieved its strongest results for memory-bound and mixed workloads. For memory-bound workloads, PA-HGS reduced average latency, queueing delay, energy-delay product, and deadline violations compared with the other policies while consuming energy similar to DVFS-only scheduling. For mixed workloads, PA-HGS achieved the lowest average latency, highest throughput, lowest energy-delay product, and lowest deadline-violation rate. These results are consistent with the modeled behavior of memory-bound phases, for which increasing core frequency provides limited performance improvement. PA-HGS can therefore assign such phases to lower-frequency or more energy-efficient configurations without substantially increasing execution time.

For compute-bound workloads, DVFS-only scheduling achieved the lowest energy consumption, whereas PA-HGS achieved the highest throughput and lowest deadline-violation rate. PA-HGS did not achieve the lowest average latency or energy-delay product for this workload class because compute-intensive phases benefit more from high-performance GPUs and higher frequencies. This result demonstrates the tradeoff between reducing instantaneous power and completing compute-intensive work quickly. A policy that aggressively lowers frequency may reduce energy consumption while increasing completion time enough to produce a less favorable energy–latency outcome.

The results also demonstrate the importance of coordinating workload phase information with heterogeneous GPU placement and frequency selection. Assigning every phase to the first available GPU can lead to inefficient execution when the device is poorly matched to the phase's computational or memory requirements. PA-HGS instead evaluates GPU suitability, frequency, queue position, and cluster power availability together. This coordination can improve energy-delay efficiency, but it also increases decision-making complexity and may introduce frequency-transition, migration, and scheduling overhead. The benefits of heterogeneous placement therefore depend on the accuracy of the performance and power models and on whether the improvement in execution time justifies any additional queueing or transition cost.

The energy-delay product is useful for evaluating these tradeoffs because it captures both energy consumption and completion time. The results show that minimizing energy alone does not necessarily produce the best overall policy. In particular, PA-HGS consumed slightly more energy than DVFS-only scheduling for memory-bound and mixed workloads but achieved lower energy-delay product values because it reduced latency more substantially. The five-percent completion-time constraint and deadline-violation metric further prevent the simulator from treating large performance losses as acceptable energy savings.

These conclusions are limited by the assumptions of the analytical models and event-driven simulation. The evaluation uses synthetic workload traces, normalized GPU parameters, and known workload-phase characteristics rather than measurements from a physical cluster. Detailed multi-tenant memory-bandwidth contention, thermal feedback, network communication, GPU memory capacity, and transient power behavior are not fully modeled. Consequently, the results should be interpreted as modeled estimates under the selected hardware, workload, power-budget, and latency assumptions.

Future work should calibrate the simulator using measurements from physical GPU clusters and real workload traces. Additional studies should evaluate phase-prediction errors, more GPU architectures, thermal effects, detailed resource interference, and larger cluster configurations. Future research should also investigate combinations of PA-HGS with other scheduling mechanisms, such as backfilling, power capping, and fair-share scheduling, to determine whether these

techniques provide complementary benefits or introduce conflicting decisions and additional scheduling overhead.

VI. Conclusion

This paper presented a model-based evaluation of energy- and latency-aware scheduling in heterogeneous GPU clusters. The proposed Phase-Aware Heterogeneous GPU Scheduler (PA-HGS) jointly selects GPU placement, operating frequency, and queue position for each workload phase under a fixed cluster-wide power budget and completion-time constraint. The framework compares PA-HGS with backfilling, workload-aware scheduling, and DVFS-only scheduling using consistent workload traces, GPU configurations, and analytical execution-time and power models.

The results show that no single policy is optimal for every workload class and metric. DVFS-only scheduling generally achieved the lowest total energy by reducing dynamic power, but it sometimes increased latency and deadline violations. PA-HGS achieved its strongest results for memory-bound and mixed workloads, where it produced lower latency, lower energy-delay products, higher throughput, and fewer deadline violations than the comparison policies. For compute-bound workloads, PA-HGS achieved the highest throughput and lowest violation rate, although DVFS-only scheduling consumed less energy and produced a more favorable energy-delay product. These findings demonstrate that workload phase, GPU heterogeneity, frequency selection, queue position, and power constraints must be considered together when evaluating energy efficiency.

Because the evaluation relies on analytical models, synthetic workload traces, and normalized GPU parameters, the conclusions represent modeled estimates rather than direct measurements from a physical GPU cluster. Future work should validate PA-HGS using real GPU clusters and workload traces, calibrate the power and performance models with hardware measurements, incorporate more detailed memory-bandwidth, and evaluate uncertainty in phase classification. Additional studies should also investigate combining PA-HGS with backfilling, power capping, and fair-share scheduling. Despite these limitations, the framework provides a reproducible basis for analyzing energy–latency tradeoffs in heterogeneous GPU systems.

References

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, doi: 10.1145/3381831.

[2] J. Fernandez, C. Na, V. Tiwari, Y. Bisk, S. Luccioni, and E. Strubell, "Energy Considerations of Large Language Model Inference and Efficiency Optimizations," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds., Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 32556–32569. doi: 10.18653/v1/2025.acl-long.1563.

[3] O. Cankur, B. Austin, D. Kulkarni, and A. Bhatele, "Characterizing Production GPU Workloads using System-wide Telemetry Data," in *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2026, pp. 899–912. doi: 10.1109/IPDPS65963.2026.00078.

[4] A. W. Mu'alem and D. G. Feitelson, "Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 6, pp. 529–543, Jun. 2001, doi: 10.1109/71.932708.

[5] Z. Ye *et al.*, "ASTRAEA: A Fair Deep Learning Scheduler for Multi-Tenant GPU Clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2781–2793, Nov. 2022, doi: 10.1109/TPDS.2021.3136245.

[6] F. Lettich, E. Carlini, F. M. Nardini, R. Perego, and S. Trani, "Power- and Fragmentation-Aware Online Scheduling for GPU Datacenters," in *2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, May 2025, pp. 43–52. doi: 10.1109/CCGRID64434.2025.00015.

[7] D. Deng, Y. Ma, Y. Wu, and H. Ma, "Fragmentation-Aware and Efficiency-Oriented Scheduling for GPU Sharing Workloads," *IEEE Trans. Cloud Comput.*, vol. 14, no. 1, pp. 242–258, Jan. 2026, doi: 10.1109/TCC.2026.3658199.

[8] M. Smith, L. Zhao, J. Cordova, X. Jiang, and M. Ebrahimi, "Energy-Efficient GPU-Intensive Workload Scheduling for Data Centers," in *2023 International Conference on Machine Learning and Applications (ICMLA)*, Dec. 2023, pp. 1735–1740. doi: 10.1109/ICMLA58977.2023.00263.

[9] Y. Luo *et al.*, "Scheduling Deep Learning Jobs in Multi-Tenant GPU Clusters via Wise Resource Sharing," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, Jun. 2024, pp. 1–10. doi: 10.1109/IWQoS61813.2024.10682877.

[10] M. V. Duc, N. D. Duong, N. D. Huy, N. D. Tuyen, N. T. Hung, and N. H. Thanh, "Energy-Aware Distributed Data-Center GPU Scheduling via Constrained Hybrid SAC with Adaptive Frequency," in *2026 40th International Conference on Information Networking (ICOIN)*, Jan. 2026, pp. 370–375. doi: 10.1109/ICOIN68469.2026.11480581.

[11] J. Yu, J. Kim, and E. Seo, "A DNN Inference Latency-aware GPU Power Management Scheme," in *2021 IEEE 3rd Eurasia Conference on IOT, Communication and Engineering (ECICE)*, Oct. 2021, pp. 551554. doi: 10.1109/ECICE52819.2021.9645654.

[12] C. V. K, Subrahmanya. K. N, K. B. S, and C. S. H. Kumar, "Runtime Power Configuration Using Frequency Voltage Scaling," in *2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS)*, Nov. 2025, pp. 1–5. doi: 10.1109/CSITSS67709.2025.11294972.

[13] Z. Wang *et al.*, "Using Analytical Performance/Power Model and Fine-Grained DVFS to Enhance AI Accelerator Energy Efficiency," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, in ASPLOS '25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 1118–1132. doi: 10.1145/3669940.3707231.

[14] X. Zhang, D. Niu, X. Liu, B. Deng, and Q. Ai, "Deep learning-based data centre energy efficiency analysis and optimisation research," in *2025 7th International Conference on Energy Systems and Electrical Power (ICESEP)*, Jun. 2025, pp. 1032–1036. doi: 10.1109/ICESEP66633.2025.11155658.

[15] P. C. M. Krishnaram, "Energy Modeling and Optimization of Multi-GPU AI Servers: A comparative Study," in *2026 IEEE Green Technologies Conference (GreenTech)*, Mar. 2026, pp. 1–6. doi: 10.1109/GreenTech68285.2026.11471623.

[16] L. Carpentieri, A. De Caro, M. S. Beni, K. Fan, and B. Cosenza, "Phase-Based Frequency Scaling for Energy-Efficient Heterogeneous Computing," in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, Jun. 2025, pp. 824–836. doi: 10.1109/IPDPS64566.2025.00078.

[17] J. Chen and M. Pericàs, "DEFT: Joint Task Placement and DVFS for Energy-Efficient Multi-GPU Runtimes," in *Proceedings of the 40th ACM International Conference on Supercomputing*, in ICS '26. New York, NY, USA: Association for Computing Machinery, Jul. 2026, pp. 1115–1127. doi: 10.1145/3797905.3807847.

[18] H. Jin, S. Li, and M. A. Gregory, "GenSched: Phase-Aware Generative Scheduling for LLM Inference in Heterogeneous Edge Networks," in *IEEE INFOCOM 2026 - IEEE Conference on Computer Communications*, May 2026, pp. 1–6. doi: 10.1109/INFOCOM59046.2026.11571651.

[19] O. Basit, Y. Liu, Z. J. Kong, and Y. C. Hu, "DualScale: Energy-Efficient Disaggregated LLM Serving via Phase-Aware Placement and DVFS," Apr. 03, 2026, *arXiv*: arXiv:2602.18755. doi: 10.48550/arXiv.2602.18755.

[20] Z. Jiang *et al.*, "HeShare: Energy-Aware and Efficient Multi-Task GPU Sharing in Heterogeneous GPU-Based Computing Systems," *IEEE Trans. Comput.*, vol. 75, no. 3, pp. 776–787, Mar. 2026, doi: 10.1109/TC.2025.3628924.

[21] G. Rattihalli *et al.*, "Fine-Grained Heterogeneous Execution Framework with Energy Aware Scheduling," in *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 35–44. doi: 10.1109/CLOUD60044.2023.00014.

[22] Y. Bao, Y. Peng, and C. Wu, "Deep Learning-Based Job Placement in Distributed Machine Learning Clusters With Heterogeneous Workloads," *IEEEACM Trans. Netw.*, vol. 31, no. 2, pp. 634–647, Apr. 2023, doi: 10.1109/TNET.2022.3202529.